



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Sviluppo di un Middleware basato su
Spring
per l'Integrazione di Flussi di Dati

Leonardo Baldo

Anno Accademico 2024-2025

- Stage presso **SyncLab S.r.l.**
- Soluzioni informatiche, ricerca e sviluppo, *system integrator*
- Progetto: studio **Middleware** basato su Spring

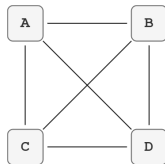


Perché serve un Middleware?



Problema:

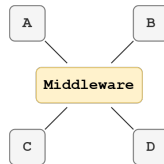
- **Connessioni** dirette con ogni sistema
- Ognuno deve **conoscere** il formato dati dell'altro



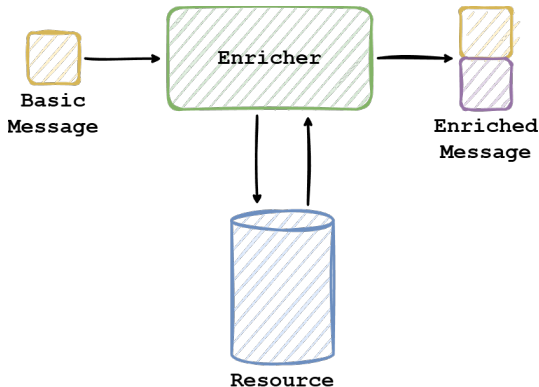
Senza Middleware

Soluzione:

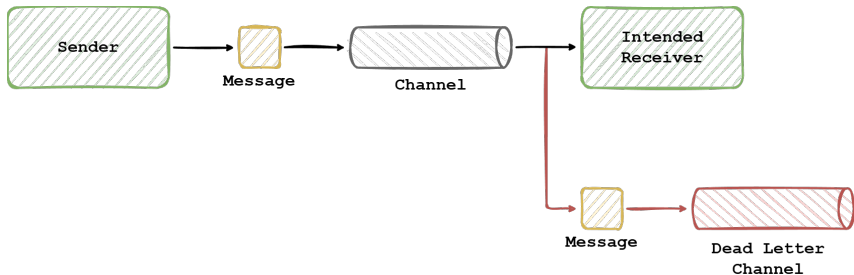
- Sistemi **eterogenei** che non comunicano direttamente
- **Disaccoppiamento** tra produttori e consumatori



Con Middleware



Content Enricher: arricchisce il messaggio



Dead Letter Channel: isola gli errori

Linguaggio e piattaforma



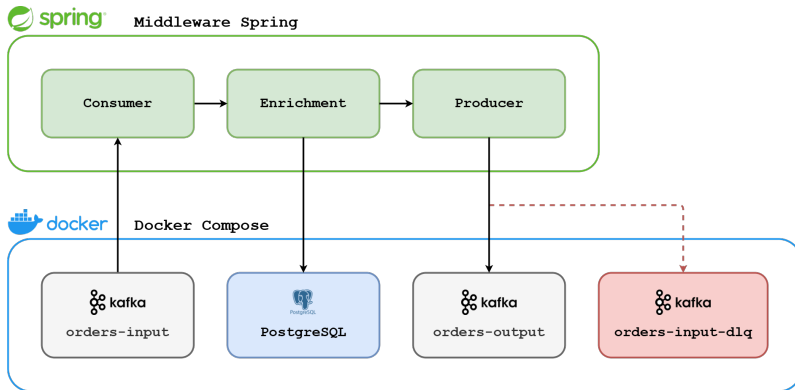
Framework applicativo



Infrastruttura di supporto



Il progetto



Jakarta Persistence:

- **@Entity**
- **@Table, @Id, @Column**

Lombok:

- **@Getter, @Setter**
- **@NoArgsConstructor**
- **@AllArgsConstructor**

```
@Entity
@Table(name = "customers")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Customer {
    @Id
    private Long id;
    @Column(name = "full_name",
            nullable = false)
    private String fullName;
    @Column(name = "email")
    private String email;
    @Column(name = "city")
    private String city;
}
```

- **Docker:** Topic Kafka (KRaft), PostgreSQL, Kafka UI

Topic	Partizioni	Ruolo
orders-input	3	ingresso
orders-output	3	uscita
orders-input-dlq	1	errori

- Obiettivo **desiderabile**, ma non obbligatorio
- Modello **stream processing** nativo e continuo
- Elabora **flussi continui** di dati
- Studiato a livello **teorico**, non implementato



Apache Flink

- **Throughput:** messaggi arricchiti per secondo
- **Latenza end-to-end:** tempo tra evento grezzo e arricchito
- **Consumer lag:** quanto il sistema *rincorre*
- **Costo interrogazione:** tempo di risposta fonte dati

- Sistemi **disaccoppiati**, facilmente modificabili
- Nessuna **perdita** di dati grazie a Kafka
- Errori **isolati** tramite Dead Letter Channel
- Ambiente **riproducibile** e facilmente replicabile

Limiti:

- **Apache Flink** non implementato, solo progettato
- **Test di performance** non eseguiti concretamente

Prossimo passo: eseguire quanto già progettato

Grazie per l'attenzione

Domande?